

AI/ML Core Fundamentals

Use as a cookbook for your AI projects

Yujia Chen

Research Syllabus

Phase 1: Core Knowledge & Foundation Lectures

Lecture 1: Deep Learning Foundations I

- **Core Focus:** The complete Machine Learning training lifecycle.
- **Key Topics:** Data preparation, forward propagation, loss functions, backpropagation, and common optimizers (e.g., SGD, Adam).

Lecture 2: Deep Learning Foundations II & Academic Reading

- **Core Focus:** Advanced ML training concepts and literature review skills.
- **Key Topics:** Common neural network architectures, and **Paper Structure Learning**.

Lecture 3: Transformer Deep Dive & Coding

- **Core Focus:** The backbone of Large Language Models and modern AI.
- **Key Topics:** Deep dive into the classic Transformer paper (*Attention Is All You Need*), followed by hands-on coding (e.g., implementing the Multi-Head Attention mechanism).

Lecture 4: Introduction to Diffusion Models

- **Core Focus:** Fundamentals of state-of-the-art generative AI.
- **Key Topics:** Basic principles of Diffusion Models (forward noise addition and reverse denoising processes), and Image Inversion techniques.

Lecture 5: Infrastructure & Practical Deployment

- **Core Focus:** Compute deployment and open-source ecosystems.
- **Key Topics:** Server setup (Linux environment, Conda, CUDA configuration), and running simple pre-trained model code using Hugging Face.

Lecture 6: Advanced Theory: Math Behind Diffusion

- **Core Focus:** The theoretical and mathematical underpinnings of diffusion.
- **Key Topics:** In-depth derivations of the complex math driving diffusion models to lay the groundwork for modifying models in your research.

Research Syllabus

Phase 2: Research Implementation & Experiments

Estimated Duration: 1 - 2 Months

After completing the core lectures, the focus will shift entirely to hands-on research, directly applying the skills learned in the first phase (especially the server setup from Lecture 5).

- **Ideation & Baseline:** Finalize a specific research direction/topic. Set up the codebase on the server and successfully run baseline models.
- **Research Implementation:** Write and modify the code to implement your novel ideas.
- **Experiments & Ablation Studies:** Run extensive comparative experiments to validate your proposed method. Conduct ablation studies to prove the contribution of individual modules, and collect/visualize the core data.

Phase 3: Paper Writing & Submission

Final Phase

- **Drafting:** Apply the paper structure principles learned in Lecture 2 to organize your experimental results and methodology into a standard academic format.
- **Revision & Polishing:** Refine your figures and tables, sharpen the storytelling in the Introduction and Abstract, and ensure academic rigor.
- **Formatting & Submission:** Format the final manuscript (usually via LaTeX) according to the guidelines of the target conference and submit.

Course Overview & Curriculum Roadmap

Module 01: Normalization

What it does: Stabilizes activation distributions across layers to enable faster, deeper training.

Topics: Batch, Layer, Group, Instance Norm, RMSNorm & Pre/Post-LN architectures.

Module 02: Activations

What it does: Introduces non-linearity so networks learn complex boundaries and control gradient flow.

Topics: Sigmoid, Tanh, ReLU, LeakyReLU, GELU, SiLU, and SwiGLU gating.

Module 03: Losses & Metrics

What it does: Defines optimization targets and measures real-world model reliability.

Topics: MSE, MAE, Huber, Focal Loss, Confusion Matrix, ROC vs. PR curves.

Module 04: Regularization & Optimization

What it does: Prevents overfitting to unseen data and navigates high-dimensional loss landscapes efficiently.

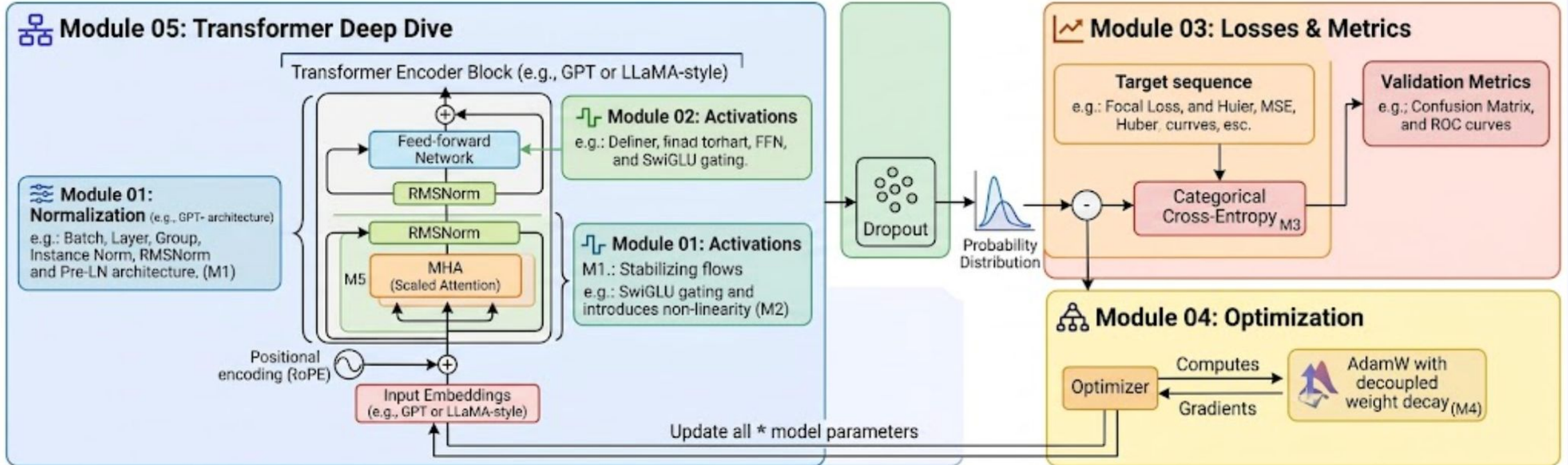
Topics: L1/L2, Inverted Dropout, SGD Momentum, Adam vs. AdamW decoupled decay.

Module 05: Transformer Deep Dive

What it does: Models dynamic token relationships at scale, replacing sequential recurrence.

Topics: Scaled Attention, MHA/MQA/GQA, RoPE positioning, $O(N^2)$ complexity & KV cache.

Course Overview & Curriculum Roadmap



Module 01 Overview: Normalization Techniques

🎯 Why Normalization Matters

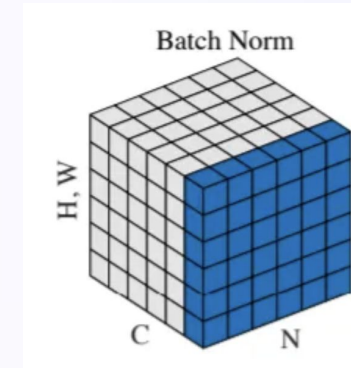
Primary Function: Controls activation variance as inputs pass through deep layers, mitigating *Internal Covariate Shift* and **exploding/vanishing gradients**.

- **Accelerates Convergence:** Allows 10x higher learning rates without instability.
- **Smooths Loss Surfaces:** Eliminates sharp gradient ravines for cleaner optimization trajectories.

🗺️ Section Roadmap

- **Batch Norm (BN):** Normalizes across mini-batch items (Vision benchmark).
- **Layer Norm (LN):** Normalizes across features per sample (Transformer standard).
- **Group Norm (GN) & Instance Norm (IN):** Micro-batch vision & style transfer.
- **RMSNorm & Pre/Post-LN:** Modern LLM speedup & deep residual stability.

Batch Normalization (BN)



What & Why

Definition: Normalizes layer activations across the *batch dimension* (N) for each channel independently.

Why Use It? Prevents *Internal Covariate Shift*. Smooths loss landscape, allowing higher learning rates.

$$\hat{x}_j^{(i)} = \frac{x_j^{(i)} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

$$y_j^{(i)} = \gamma_j \hat{x}_j^{(i)} + \beta_j$$

Learnable scale (γ) and shift (β) parameters preserve original expressiveness.

Pros, Cons & Use Cases

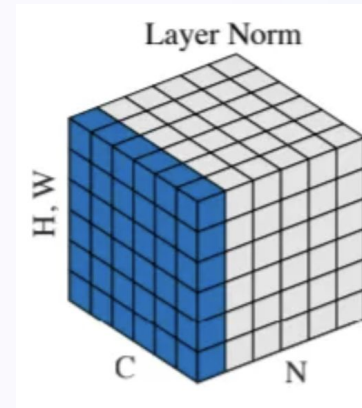
Pros:

- Acts as a light regularizer by injecting mini-batch noise.
- Highly effective in convolutional vision models (CNNs).

Cons & Limitations:

- Fails on small mini-batches (unstable mean/variance estimation).
- Requires tracking exponential running statistics for inference.
- Leaks information across batch items (unusable in dynamic RNNs/LLMs).

Layer Normalization (LN)



What & Why

Definition: Normalizes activations across the *feature/channel dimensions* for each individual sample independently.

Why Use It? Completely decouples normalization from batch size. Ideal for sequence models (NLP & Transformers) with dynamic sequence lengths.

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta$$

Mean and variance are computed independently per token/sample.

Pros, Cons & Applications

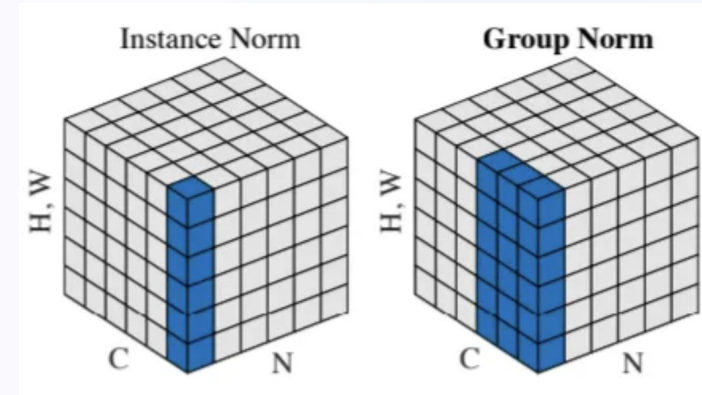
Pros:

- Identical behavior during training and inference (no running stats required).
- Operates perfectly with mini-batch size of 1 or dynamic padding lengths.

Cons & Key Takeaways:

- Slightly underperforms BN in vision CNNs due to non-shared spatial statistics.
- Foundational standard in BERT, GPT, and classic Transformer blocks.

Group (GN) & Instance (IN) Norm



Group Normalization (GN)

Mechanics: Divides channels into G groups and normalizes within each group per sample.

Why Use It? Bridges BN and LN for computer vision tasks.

- **Ideal Scenario:** High-resolution vision tasks (Object Detection, Segmentation) where batch size is 1 or 2 per GPU.
- **Benefit:** Maintains high accuracy independent of batch size.

Instance Normalization (IN)

Mechanics: Normalizes strictly across spatial dimensions (H, W) per channel per sample.

Why Use It? Strips style-specific contrast and luminance from individual images.

- **Primary Application:** Style Transfer, CycleGANs, Image Generation.
- **Intuition:** Removes style while preserving underlying structural content.

RMSNorm & Pre-LN vs Post-LN

RMSNorm (Root Mean Square Norm)

Core Concept: Drops mean-centering in LayerNorm and scales activations strictly by root mean square:

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad \text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma$$

LLM Standard: Used in LLaMA, Mistral, and Gemma to reduce compute overhead by 7–10% without loss of accuracy.

Pre-LN vs. Post-LN Architecture

Post-LN (Original Transformer):

Output = LN(x + SubLayer(x)). Gradients vanish at deep layers; requires strict learning rate warmup.

Pre-LN (Modern Standard):

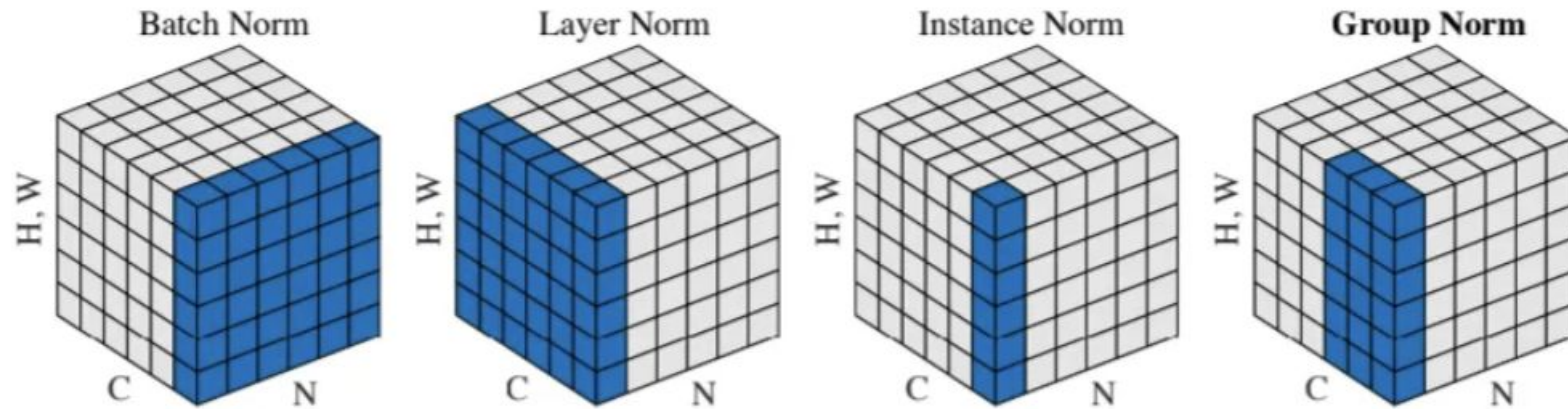
Output = x + SubLayer(LN(x)). Preserves an unobstructed identity residual highway; enables ultra-deep network stability.

Visual Comparison of Normalization Layers

Understanding Tensor Dimensions : (N , C , H , W)

- **Batch Norm:** Normalizes vertical slices across the Batch dimension N .
- **Layer Norm:** Normalizes horizontal slices across Channels & Spatial dims C , H , W .
- **Instance Norm:** Normalizes single-channel spatial blocks H , W .
- **Group Norm:** Normalizes grouped sub-channels per sample independently.

Rule of Thumb: BN for large-batch CNNs, LN/RMSNorm for Transformers, GN for micro-batch vision, IN for Style Transfer.



Knowledge Check: Normalization

Q1: Why do modern LLMs like LLaMA prefer Pre-LN/RMSNorm over Post-LN LayerNorm?

- A) Post-LN increases memory consumption by 4x during backpropagation.
- B) Pre-LN maintains a clean residual gradient path; RMSNorm removes mean calculation to boost speed.
- C) Pre-LN allows Batch statistics to leak across attention tokens.
- D) RMSNorm eliminates the need for Positional Embeddings.

Correct Answer: B. Pre-LN prevents vanishing gradients in deep transformers by maintaining an unobstructed residual path, while RMSNorm saves compute by skipping mean-centering.

Q2: When training an Object Detector on 4K resolution images with mini-batch size = 2 per GPU, which norm is best?

- A) Batch Normalization (BN)
- B) Group Normalization (GN)
- C) Instance Normalization (IN)
- D) RMSNorm

Correct Answer: B. BN yields inaccurate mean/variance estimates at a mini-batch size of 2. Group Norm divides channels into groups and normalizes independently of batch size.

Module 02 Overview: Activation Functions

🎯 Why Non-Linearities Exist

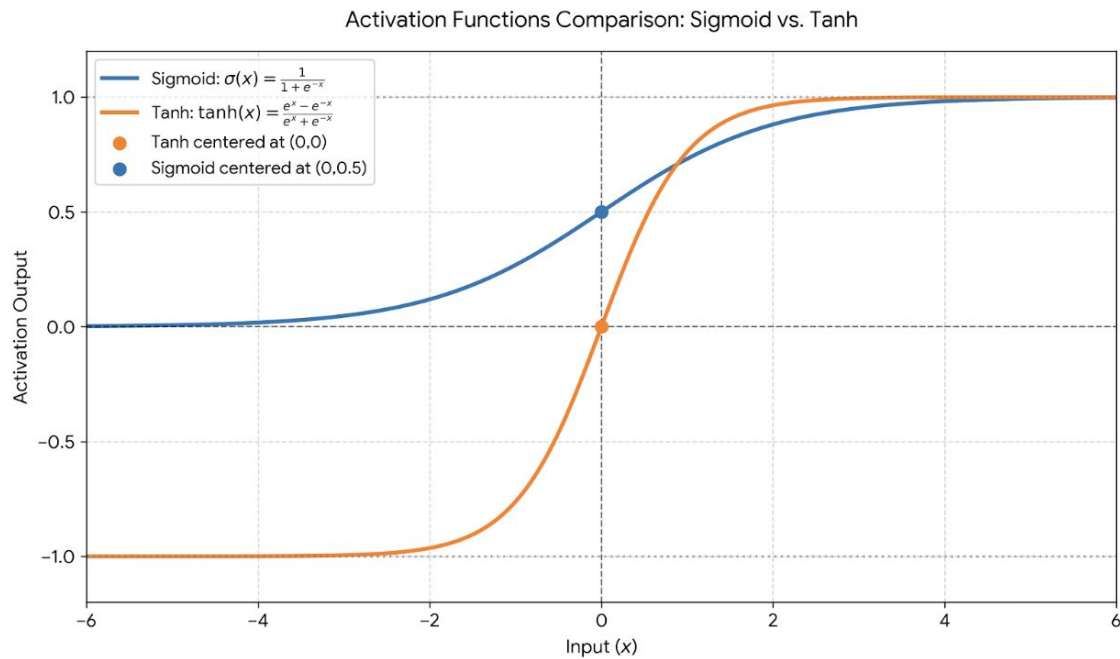
Primary Function: Without activation functions, stacking linear layers $W \cdot x$ mathematically collapses into a single linear matrix multiplication regardless of depth!

- **Expressiveness:** Enables neural networks to approximate arbitrary continuous functions (Universal Approximation Theorem).
- **Gradient Control:** Directly influences whether gradients explode, vanish, or flow smoothly back during SGD.

📖 Section Roadmap

- **Sigmoid & Tanh:** Early saturating functions & saturation limits.
- **ReLU & LeakyReLU:** Piecewise linear solutions & dying neuron fixes.
- **GELU & SiLU:** Smooth probabilistic non-monotonic curves in BERT & LLaMA.
- **SwiGLU Gating:** 3-matrix gated MLP blocks in state-of-the-art LLMs.

Sigmoid & Tanh Curves & Mechanics



Properties & Limitations

Sigmoid:

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

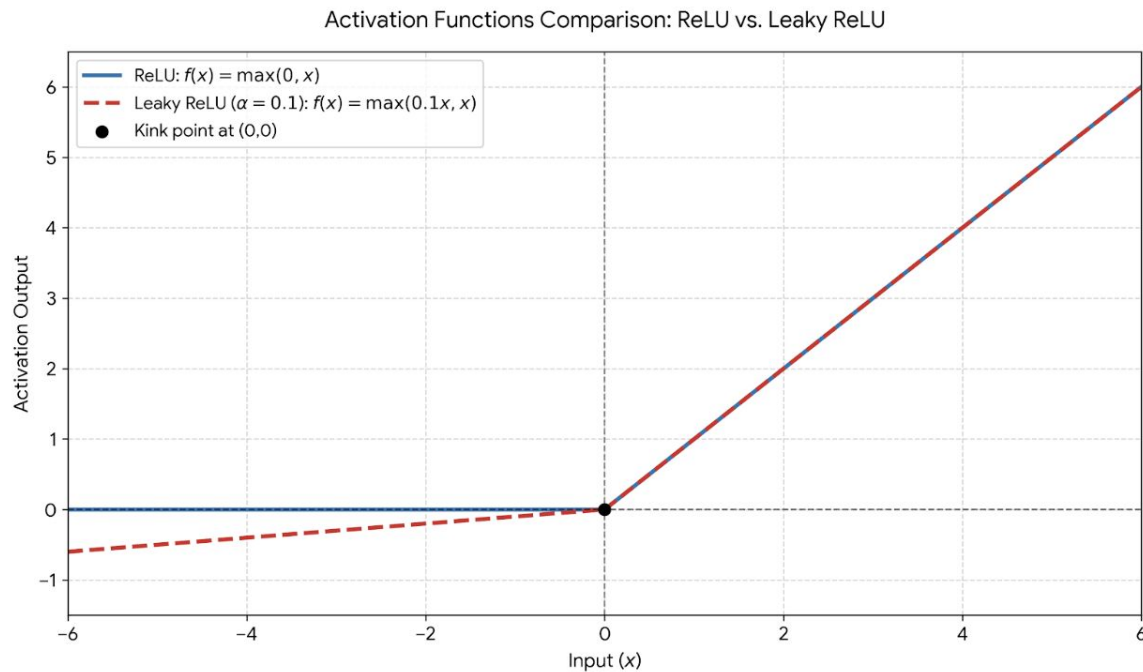
- Max derivative is 0.25 at $x=0$, leading to severe **vanishing gradients** in deep models.

Tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Zero-centered (faster gradient convergence than Sigmoid).
- Saturates at extremes $\tanh'(x) \rightarrow 0 \quad |x| \gg 0$

ReLU & LeakyReLU: Solving Vanishing Gradient



Mechanics & Dying ReLU Problem

ReLU:

$$f(x) = \max(0, x)$$

Derivative is 1 for $x > 0$ and 0 for $x < 0$.

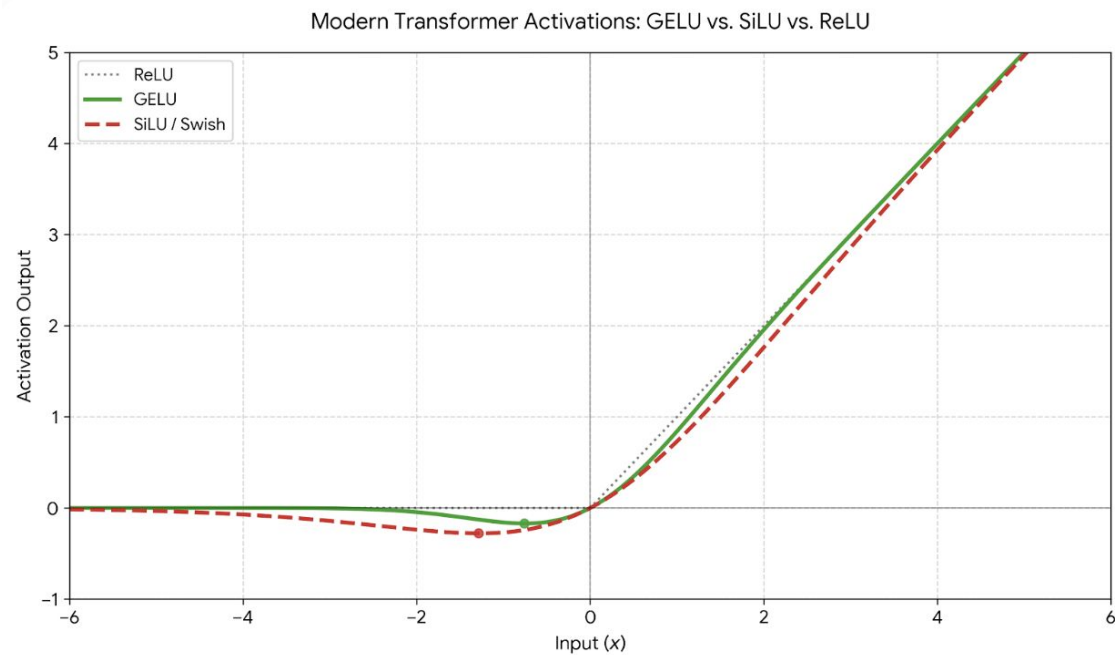
- **Dying ReLU Problem:** Large negative gradients set weights such that neuron always outputs 0. Gradient stays forever 0.

LeakyReLU Solution:

$$f(x) = \max(\alpha x, x) \quad (\alpha = 0.01)$$

- Provides a small positive slope for $x < 0$, keeping gradient flow active and preventing dead neurons.

Smooth & Gated Activations: GELU, SiLU, SwiGLU



Modern Transformer Standards

GELU:

$$\text{GELU}(x) = x \cdot \Phi(x) \approx x \cdot \sigma(1.702x)$$

Smooth probabilistic gating used in BERT, GPT-2/3, ViT.

SiLU / Swish:

$$f(x) = x \cdot \sigma(x)$$

Standard in EfficientNet & LLaMA.

SwiGLU (Gated Linear Unit):

$$\text{SwiGLU}(x) = (xW_1 \cdot \sigma(xW_1)) \otimes (xW_2)$$

Uses 3 weight matrices in Transformer MLP blocks, greatly increasing capacity and state-of-the-art accuracy.

Knowledge Check: Activations

Q1: Why does GELU perform better than ReLU in deep Transformer models?

- A) GELU guarantees strict zero outputs for all negative inputs.
- B) GELU provides a smooth, probabilistic non-monotonic curvature around zero, preserving small gradient updates.
- C) GELU replaces matrix multiplication with fast additions.
- D) GELU prevents attention weights from summing to 1.

Correct Answer: B. Unlike ReLU's sharp non-differentiable drop at 0, GELU smoothly interpolates inputs based on their magnitude probabilistically, preventing dead neurons.

Q2: What structural change occurs when switching a Transformer MLP from standard ReLU to SwiGLU?

- A) The hidden dimension must be multiplied by 4x.
- B) It uses two parallel linear projections (one gated by Swish) element-wise multiplied with a third projection.
- C) Softmax is added after every linear transformation.
- D) Attention heads are merged into a single query head.

Correct Answer: B. SwiGLU introduces a gated architecture with 3 weight matrices in the Feed-Forward Network instead of the traditional 2 linear layers.

Module 03 Overview: Loss Functions & Metrics

🎯 Defining Objectives & Evaluation

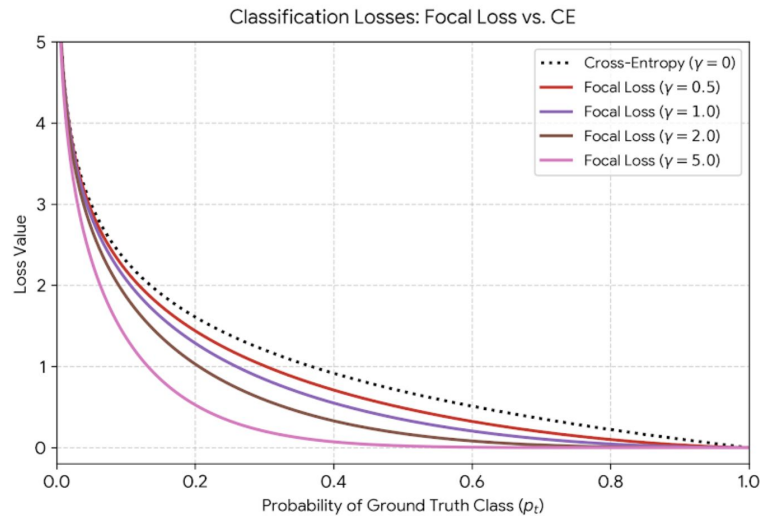
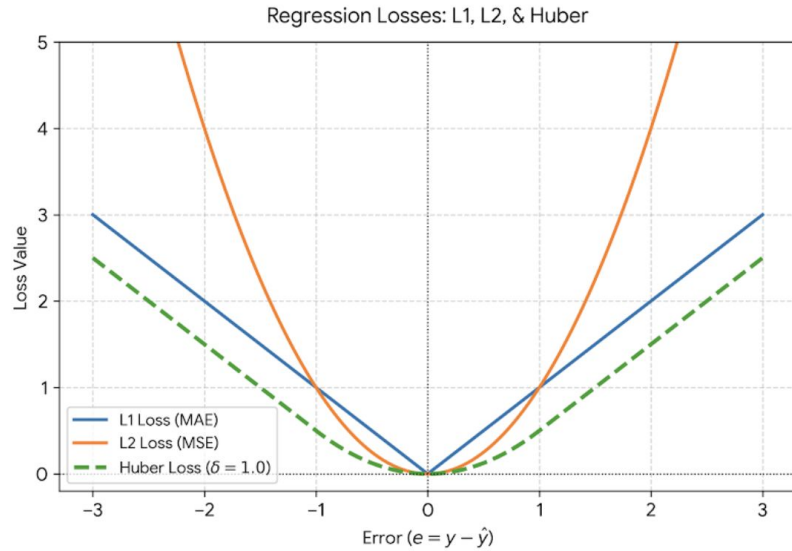
Primary Function: Loss functions provide a differentiable signal for SGD during training, while evaluation metrics quantify real-world performance under constraints (e.g., class imbalance).

- **Optimization vs Evaluation:** Losses must be smooth/differentiable; metrics (F1, mAP) reflect business impact.
- **Robustness:** Choosing the right loss/metric prevents models from exploiting class shortcuts.

🗺️ Section Roadmap

- **Regression Losses:** MSE (L_2), MAE (L_1), and Huber loss sensitivity.
- **Classification Losses:** Cross-Entropy & Focal Loss for dense imbalance.
- **Confusion Matrix:** Deconstructing TP, FP, TN, FN & Type I/II errors.
- **ROC vs PR Curves:** Choosing the right curve under massive imbalance.

Loss Functions: MSE, MAE, Huber & Focal Loss



Key Formulations

MSE (L₂):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Extremely sensitive to outliers due to squaring.

MAE (L₁):

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Robust to outliers, but derivative at 0 is undefined.

Focal Loss (Class Imbalance):

$$\text{FL} = -\alpha_t (1 - p_t)^\gamma \log(p_t)$$

Down-weights easy background examples ($p_t \approx 1$), focusing updates on hard targets.

Confusion Matrix: TP, FP, TN, FN & Formulas

Confusion Matrix Breakdown

		Actual Class	
		Positive (1)	Negative (0)
Predicted	Positive (1)	True Positive (TP) Predicted + / Actual +	False Positive (FP) Type I Error
	Negative (0)	False Negative (FN) Type II Error	True Negative (TN) Predicted - / Actual -

Core Metric Formulas

Accuracy:

$$\text{Accuracy} = \frac{TP+TN}{TP + TN + FP + FN}$$

Precision & Recall:

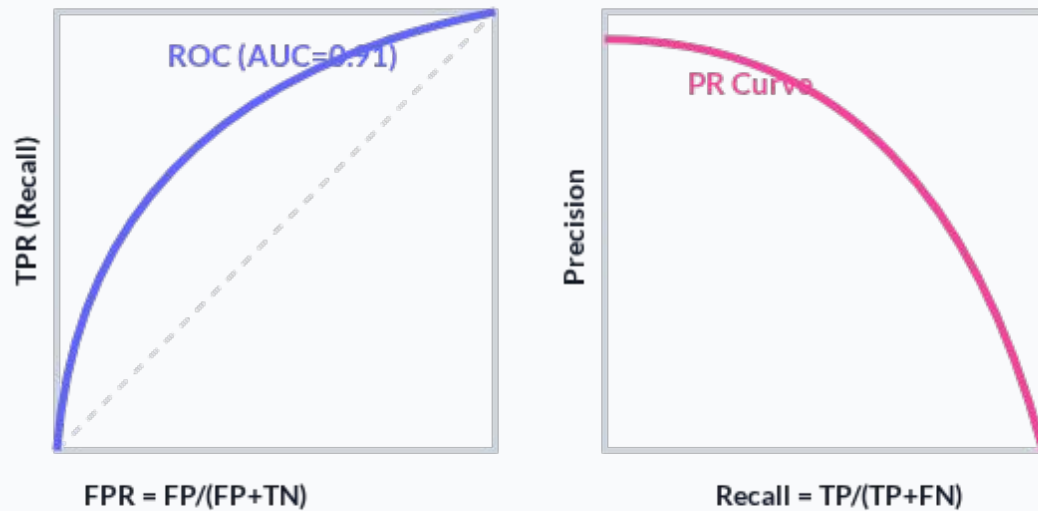
$$\text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}$$

F1-Score (Harmonic Mean):

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC Curve vs Precision-Recall (PR) Curve

ROC vs PR Curves Plot



Calculating & Choosing the Right Curve

ROC Calculation: Vary decision threshold on output probability. Plot TPR vs FPR:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

The Imbalance Trap: With 1:10000 negative ratio, huge TN keeps FPR tiny, masking false positives!

PR Curve Solution: Plots Precision vs Recall. Completely ignores TN, making it highly sensitive to False Positives on imbalanced datasets.

Knowledge Check: Losses & Metrics

Q1: Why is PR-AUC preferred over ROC-AUC when evaluating a Fraud Detection model with 99.9% negative transactions?

- A) ROC-AUC cannot be calculated for binary classification tasks.
- B) ROC False Positive Rate stays small due to huge True Negatives, hiding poor precision.
- C) PR-AUC is less sensitive to decision threshold changes.
- D) Cross-Entropy loss requires PR-AUC for optimization.

Correct Answer: B. In massive class imbalance, extremely large True Negative numbers make the False Positive Rate ($FPR = FP / (FP + TN)$) negligibly small, masking false positives in ROC curves.

Q2: How does Focal Loss solve the issue of class imbalance in dense object detectors?

- A) It clips gradients of rare target classes.
- B) It uses $(1-p_t)^\gamma$ to suppress loss contribution from easy background examples.
- C) It converts multi-class classification into L2 regression.
- D) It normalizes loss across channel spatial dimensions.

Correct Answer: Focal Loss dynamically scales down the loss contribution from well-classified easy samples ($p_t \approx 1$), focusing updates strictly on hard target classes.

Module 04 Overview: Regularization & Optimization

🎯 Generalization & Loss Landscapes

Primary Function: Regularization constrains model capacity to prevent memorization, while Optimization efficiently navigates non-convex, high-dimensional loss surfaces.

- **Generalization Gap:** Minimizing the difference between training loss and test error.
- **Decoupled Decay:** Ensuring weight reduction is independent of adaptive gradient scaling.

📅 Section Roadmap

- **Regularization:** L1 (Lasso) vs. L2 (Ridge), Inverted Dropout, Spatial Dropout.
- **Optimizer Trajectories:** SGD, Momentum, RMSprop, Adam, and AdamW.
- **Decoupled Decay:** Why AdamW is required for Transformer fine-tuning.
- **Warmup Mechanics:** Preventing catastrophic early variance destruction.

Regularization Techniques: L1, L2, Dropout, Augmentation

L1 (Lasso) vs L2 (Ridge) vs ElasticNet

L1 Regularization:

$$\text{Loss} + \lambda \sum_{i=1}^d |w_i|$$

Constrains weight space with diamond contour. Forces weights to exact zero for **Sparsity** / **Feature Selection**.

L2 Regularization:

$$\text{Loss} + \lambda \sum_{i=1}^d w_i^2$$

Circular contour. Penalizes large weights smoothly, keeping features with small weights.

ElasticNet: Combines penalties $\alpha L_1 + (1 - \alpha) L_2$

Dropout, Label Smoothing & Augmentation

Inverted Dropout: Scales surviving activations by $\frac{1}{1-p}$ during training, eliminating test-time scaling.

Spatial Dropout: Drops entire feature channels in CNNs rather than individual pixels.

Label Smoothing: Converts target to smooth value: $y \in \{0, 1\}$

$$y_{\text{smooth}} = 1 - \epsilon + \frac{\epsilon}{K}$$

Optimizer Loss Landscapes & Trajectories

Loss Contour Optimization Paths

SGD Oscillations $\theta_{t+1} = \theta_t - \eta \cdot g_t$



Optimizer Evolution

SGD with Momentum: Accumulates past velocity:

$$m_t = \beta \cdot m_{t-1} + (1 - \beta) \cdot g_t \quad \theta_{t+1} = \theta_t - \eta \cdot m_t$$

Dampens high-curvature ravine oscillations.

RMSprop / AdaGrad: Scales learning rates inversely by root sum of squared gradients.

Adam & AdamW Decoupled Weight Decay:

$$w_{t+1} = w_t - \eta \lambda w_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

Decouples weight decay directly from gradient update step!

Knowledge Check: Optimization

Q1: Why is learning rate warmup mathematically required when starting Adam training for Large Transformers?

- A) Warmup initializes the model weights to orthogonal matrices.
- B) Second moment v_t estimates are inaccurate initially, causing wildly unstable early updates.
- C) Warmup prevents activation memory from overflowing GPU cache.
- D) Warmup forces learning rate to decay to zero before residual convergence.

Correct Answer: In early steps, variance v_t is estimated from very few samples. Without warmup, $\frac{m_t}{\sqrt{v_t} + \epsilon}$ produces wildly large update vectors that destroy pre-trained features.

Q2: What is the primary difference between standard L2 regularization in PyTorch Adam vs AdamW?

- A) AdamW ignores negative gradients during step execution.
- B) Adam adds decay to gradient before variance division; AdamW subtracts decay directly from weights.
- C) AdamW scales weight decay exponentially with sequence length.
- D) Standard Adam does not support momentum terms.

Correct Answer: Standard Adam adds L2 decay to the gradient vector before dividing by $\sqrt{v_t}$, suppressing decay for large-gradient weights. Decoupling weight decay in AdamW ensures weight reduction is strictly invariant to gradient scale.

Module 05 Overview: Transformer & Attention Deep Dive

🎯 Modern Sequence Paradigm

Primary Function: Replaces sequential recurrence (RNNs) with dynamic, parallelized content addressing, enabling foundation models to scale across massive contexts.

- **Context Addressing:** Q, K, V inner products measure pair-wise token relevance.
- **Scaling & Extrapolation:** Managing $O(N^2)$ memory bottlenecks & dynamic sequence length extension.

📋 Section Roadmap

- **Scaled Dot-Product:** Mathematical variance scaling $\sqrt{d_k}$.
- **Attention Variants:** MHA vs MQA vs GQA memory bandwidth trade-offs.
- **Position Embeddings:** Sinusoidal vs Absolute vs RoPE complex rotation.
- **Complexity & KV Cache:** $O(N^2)$ GPU HBM costs & decoding memory footprint.

Scaled Dot-Product Attention Mechanics

Formula & Scaling Derivation

$$\text{Attention} (Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Why Scale by $\sqrt{d_k}$?

If, dot product has mean and variance.

$$Q, K \sim N(0, 1) \quad Q \cdot K = \sum_{i=1}^{d_k} q_i k_i$$

For large d_k , variance becomes large, pushing Softmax into extreme regions with near-zero gradients. Dividing by $\sqrt{d_k}$ scales variance back to 1.

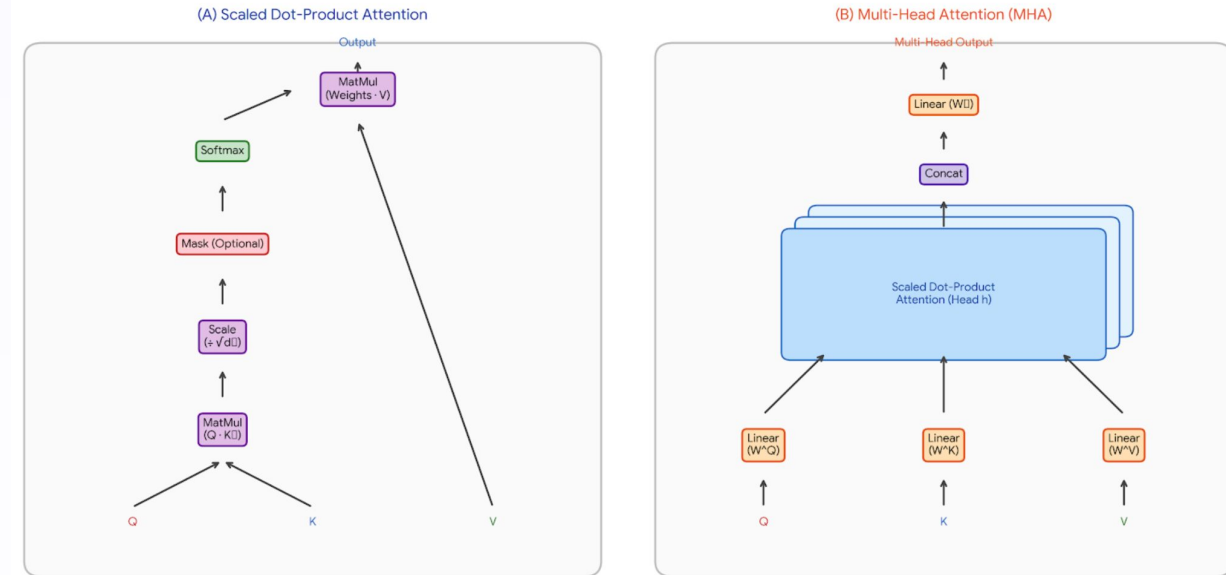
Multi-Head Attention (MHA)

Allows the model to jointly attend to information from different representation subspaces at different positions.

Projection: Q, K, V are linearly projected h times into smaller dimension.

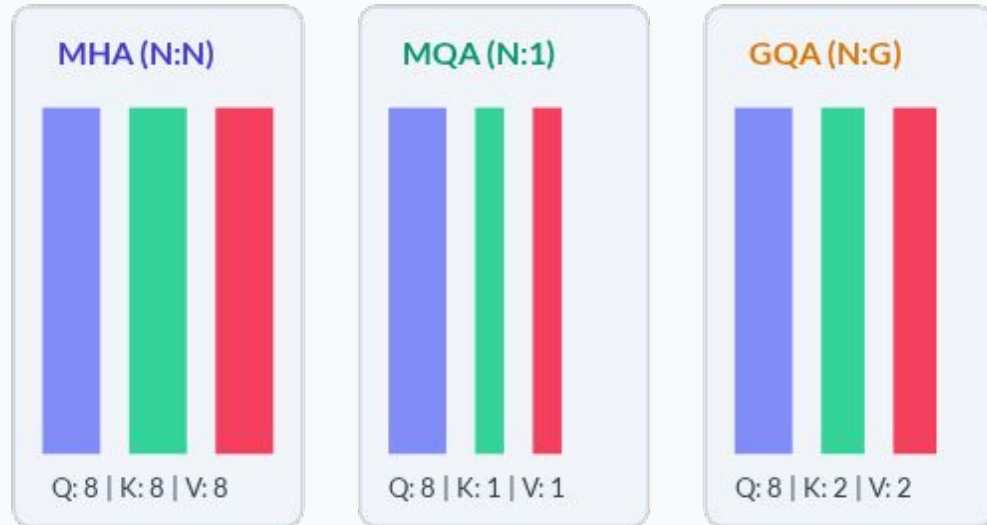
$$d_v = d_{\text{model}} / h$$

Outputs: Concat(head1, ..., headh) W_O .



MHA vs MQA vs GQA Architecture Comparison

Visual Head Breakdown



Trade-off Analysis

Multi-Head (MHA): High quality, high KV-cache memory bandwidth bottleneck during autoregressive decoding.

Multi-Query (MQA): All Query heads share 1 Key and 1 Value head. 8x to 64x KV cache memory reduction, but slight accuracy drop.

Grouped-Query (GQA): Gold standard in LLaMA 2/3 & Mistral. Query heads grouped into groups; balances memory speed and MHA quality.

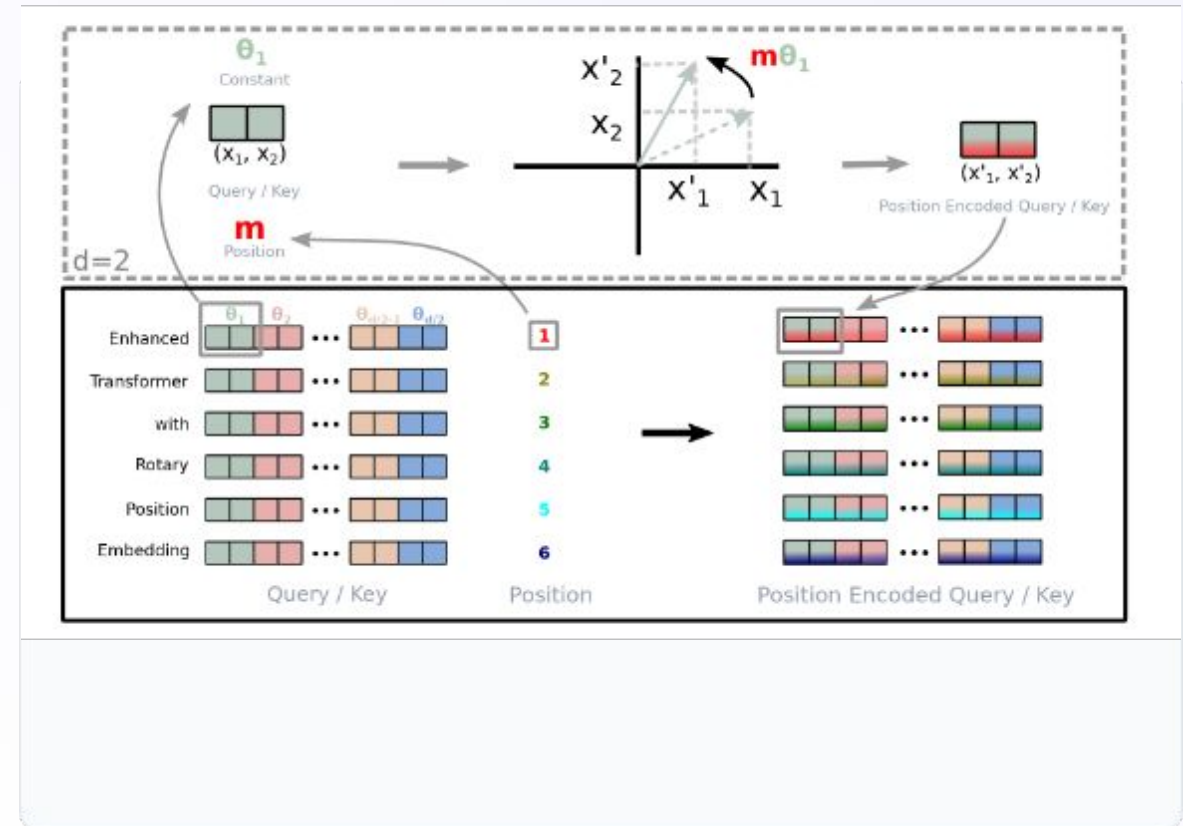
Positional Encodings & Rotary Position Embedding (RoPE)

Types of Positional Embeddings:

- **Sinusoidal (Original):** Fixed sine/cosine functions added to token embeddings.
- **Learned Absolute:** Learned vectors added per position (BERT, GPT-2). Fails beyond pre-train max length.
- **Rotary Position Embedding (RoPE):** Rotates Query and Key 2D sub-vectors in complex space by angle $m\theta$.

$$\langle R_m Q, R \rangle = Q^T R_{n-m} K$$

Why RoPE is Superior: Inner product depends strictly on relative offset $(m -)$, enabling natural length extrapolation!



Transformer Complexity & KV Cache Analysis

Self-Attention Cost $O(N^2)$

Matrix Scaling: QK^T generates an $N \times N$ attention matrix (N = sequence length).

GPU HBM Bottleneck: Reading/writing full $N \times N$ matrix from GPU HBM to SRAM limits context windows.

FlashAttention Solution: Computes exact softmax using SRAM block-tiling and online softmax re-scaling without saving $N \times N$ matrix.

KV Cache Memory Formula

During autoregressive decoding, keys and values of previous tokens are cached to avoid recomputation $O(N^3)$

Bytes per Token Formula (FP16 / 2 bytes):

$$\text{Bytes} = 2 \times 2 \times n_{\text{layers}} \times d_{\text{model}}$$

For LLaMA-70B (80 layers, $d_{\text{model}}=8192$), KV cache takes **~2.6 MB per token!**

Knowledge Check: Transformer Architecture

Q1: What is the primary mathematical reason for scaling dot-product attention scores by $1/\sqrt{d_k}$?

- A) To ensure attention weights sum to zero.
- B) To prevent dot products from exploding in variance at high dimensions, which pushes Softmax into zero-gradient regions.
- C) To convert dot product into Euclidean distance.
- D) To align Query and Key dimensions for matrix multiplication.

Correct Answer: Unscaled dot products yield variance proportional to d_k . At high dimensions, this large variance causes Softmax outputs to saturate near 1 and 0, resulting in vanishingly small gradients during backpropagation.

Q2: How does Rotary Position Embedding (RoPE) represent relative positions in attention?

- A) By adding scalar positional offsets to input token embeddings.
- B) By applying a 2D rotation matrix to Q and K vectors, making their inner product dependent strictly on relative distance $(m - n)$.
- C) By masking future token attention scores in decoder blocks.
- D) By dividing key matrices by batch size.

Correct Answer: RoPE multiplies 2D sub-pairs of Q and K by orthogonal rotation matrices proportional to position index, ensuring the resulting inner product strictly encodes the relative distance offset $(m - n)$.